

PONTIFICIA UNIVERSIDAD CATÓLICA DEL PERÚ
FACULTAD DE CIENCIAS E INGENIERÍA

ALGORITMIA Y ESTRUCTURA DE DATOS

4ta. práctica (tipo B)
(Segundo Semestre 2025)

Duración: 1h 50 min.

- **No puede utilizar apuntes, solo hojas sueltas en blanco.**
- En cada función el alumno deberá incluir, a modo de comentario, la forma de solución que utiliza para resolver el problema. De no incluirse dicho comentario, el alumno perderá el derecho a reclamo en esa pregunta.
- No puede emplear plantillas o funciones no vistas en los cursos de programación de la especialidad.
- Los programas deben ser desarrollados en el lenguaje C++. Si la implementación es diferente a la estrategia indicada o no la incluye, la pregunta no será corregida.
- Un programa que no muestre resultados coherentes y/o útiles será corregido sobre el 50% del puntaje asignado a dicha pregunta.
- Debe utilizar comentarios para explicar la lógica seguida en el programa elaborado. El orden será parte de la evaluación.
- **Solo está permitido acceder a la plataforma de PAIDEIA, cualquier tipo de navegación, búsqueda o uso de herramientas de comunicación se considera plagio por tal motivo se anulará la evaluación y se procederá con las medidas disciplinarias dispuestas por la FCI.**
- Para esta evaluación solo se permite el uso de las librerías `iostream`, `iostream`, `climits`, `cstring`, `cmath` o `fstream`
- Su trabajo deberá ser subido a PAIDEIA.
- **Es obligatorio usar como compilador CLion.**
- Los archivos deben llevar como nombre su código de la siguiente forma `codigo_LAB4_P#&` (donde # representa el número de la pregunta a resolver y & la alternativa)

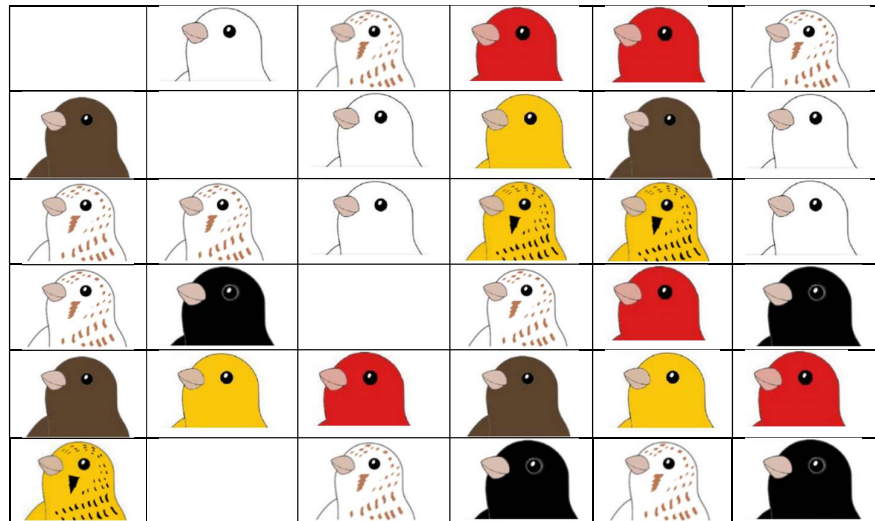
Pregunta 1 (10 puntos)

Una empresa dedicada a la importación de aves ornamentales de competencia se encarga de preparar los envíos desde el país de origen hacia el Perú, para esta tarea emplea contenedores especiales que garanticen la seguridad y salud de las pequeñas aves. Los contenedores son cajas que tienen N filas, en cada fila pueden viajar N canarios. Los criadores siempre colocan parejas de estas hermosas aves para la reproducción, desafortunadamente al momento de preparar las cajas se dan cuenta que les sobran algunas aves de diferentes mutaciones (ninguna repetida), por tal motivo es posible algunas de las filas no estén completas y algunos canarios no tengan su pareja respectiva. Luego de observar cada fila se dan cuenta que en algunas faltan como máximo un canario de una mutación determinada. Debido a que son animales vivos las aves se han movido y están en diferentes ubicaciones no necesariamente al lado de su pareja seleccionada. A continuación, se muestran las mutaciones que se pueden transportar:

Mutaciones:

Amarillo	Rojo	Blanco	Negro	Bruno	Ágata	Isabel
						

A continuación, un ejemplo de una caja de transporte de canarios donde **N** = 6.



Respuesta:



Desarrolle un programa que utilice la estrategia **divide y vencerás** para todas sus operaciones que ayude a determinar las mutaciones que faltan en cada fila, con el fin de ser completadas. La complejidad máxima de este programa debe ser $O(n^2 \cdot \log(n))$. Recuerde que todas las funciones que desarrolle deben emplear la estrategia indicada.

Pregunta 2 (10 puntos)

Un **árbol binario de cúmulos mínimo (min-heap)**, es una estructura de datos que cumple dos propiedades:

- **Estructura de árbol binario casi completo:** todos los niveles del árbol están completamente llenos, excepto el último, que se completa de izquierda a derecha.
- **Propiedad de montículo mínimo:** el valor almacenado en cada nodo es menor o igual que el valor de sus hijos. Esto significa que el nodo con **el valor más pequeño está en la raíz**.

Dada las siguientes secuencias de números a insertar, construya **paso a paso (nodo a nodo)** el árbol de Cúmulos mínimo.

- Debe **comenzar a insertar los nodos desde la raíz**, caso contrario, no se validará la pregunta.
- Use **exclusivamente** el TAD **Árbol Binario y (Cola ó Pila)**. Debe elegir entre una de las siguientes estrategias a utilizar.
 1. Use **una cola para encontrar rápidamente el siguiente nodo padre disponible** que tenga un hijo libre. La cola no guarda los datos iniciales.
 2. Use **una pila** para guardar la **combinación binaria de la posición dónde se encuentra el siguiente nodo a insertar**. Numere los nodos secuencialmente por niveles (empezando desde 1 para la raíz). La posición del siguiente nodo a insertar determina la ruta desde la raíz: Use la regla:
 - 0 = ve a la izquierda
 - 1 = ve a la derecha
- La inserción la debe realizarse **por niveles de izquierda a derecha**, es decir, buscando siempre el **primer lugar libre** en el que el nuevo nodo pueda adjuntarse como hijo de un nodo padre ya existente.
- Después de la inserción, verifique si el árbol sigue cumpliendo las propiedades de un **árbol de cúmulos mínimo**. En caso contrario, realizar los ajustes necesarios (intercambio de valores entre nodos) para **restablecer el orden** del árbol.

Explicación: Para insertar un nuevo dato, primero localizamos un nodo existente (el **padre**) que tenga libre una posición de hijo (izquierdo o derecho). El nuevo dato se convierte en un nuevo nodo que se **enlaza** a esa posición libre.

Para hacer más sencillo el proceso añada a la estructura **NodoArbolBinario** un puntero al nodo **padre**, de modo que sea más fácil realizar los intercambios necesarios cuando se deba restaurar la propiedad del cúmulo mínimo.

Secuencia de datos a insertar.

Caso 1: A = {3, 5, 7, 2, 8, 9}

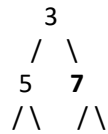
Insertar 3, el árbol está vacío, se inserta en el nivel 1, no hay nada que ordenar porque es el nodo raíz.

```
  3
 / \
```

Insertar 5, se localiza un nodo existente (el **padre**) que tenga libre una posición de hijo (izquierdo o derecho). Se inserta el 5 en el nivel 2, a la izquierda de 3. Se quiere ordenar, pero 5 es mayor que 3. Entonces, no hay nada que ordenar.

```
  3
 / \
 5
```

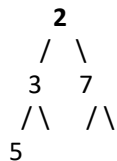
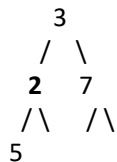
Insertar 7, se localiza un nodo existente (el **padre**) que tenga libre una posición de hijo (izquierdo o derecho). Se inserta el 7 en el nivel 2, a la derecha de 3. Se quiere ordenar, pero 7 es mayor que 3. Entonces, no hay nada que ordenar.



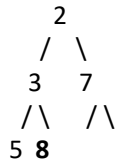
Insertar 2, se localiza un nodo existente (el **padre**) que tenga libre una posición de hijo (izquierdo o derecho). Se inserta el 2 en el nivel 3, a la izquierda de 5. Se quiere ordenar, se aplica el proceso de **heapify-up (flotar)**, se va comparando el nuevo nodo 2 con su padre el 5 y si es menor que el padre debe intercambiar el elemento, así se va comparando e intercambiando hasta que no haya intercambio.



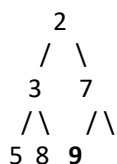
Reordenar:



Insertar 8, se localiza un nodo existente (el **padre**) que tenga libre una posición de hijo (izquierdo o derecho). Se inserta el 8 en el nivel 3, a la derecha de 3. Se quiere ordenar, pero 8 es mayor que 3. Entonces, no hay nada que ordenar.

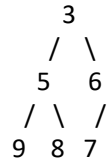


Insertar 9, se localiza un nodo existente (el **padre**) que tenga libre una posición de hijo (izquierdo o derecho). Se inserta el 9 a la izquierda de 7. Se quiere ordenar, pero 9 es mayor que 7. Entonces, no hay nada que ordenar. Terminó el proceso.

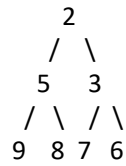


Pruebe los resultados para los casos 2 y 3.

Caso 2: $B = \{3, 5, 7, 9, 8, 6\}$



Caso 3: $C = \{3, 5, 7, 9, 8, 6, 2\}$



Al finalizar el laboratorio, comprima la carpeta de su proyecto empleando el programa Zip que viene por defecto en el Windows, **no se aceptarán los trabajos compactados con otros programas como RAR, WinRAR, 7zip o similares**. Luego súbalo a la tarea programa en Paideia para este laboratorio.

Profesores del curso:

Ana Roncal
Fernando Huamán
David Allasi
Rony Cueva
Erasmus Gómez

San Miguel, 15 de noviembre del 2025